인수인계서: 디자인 업그레이드 · 시연영상 · 검수

- 작성일: 2026-10-23(금) · 작성: 의뢰인(출품자) 쪽
- 받는 사람: 디자인 업그레이드, 3분 시연영상 제작, 검수를 맡는 분
- 프로젝트: 「전력안심망 AI」 — 2026 AI 라이프 솔루션 챌린지 출품작
- 운영 사이트: https://power-safety-net-ai.vercel.app
- 저장소: https://github.com/Haribear99/power-safety-net-ai (공개로 바꿀 예정, 0장·7장)
- **공모전 마감: 10/31(토) 24:00 KST, 이메일 제출. 의뢰인의 제출 목표일은 10/30(금)입니다.**

이 문서만 읽으면 일을 시작할 수 있게 썼습니다. 더 깊은 근거는 11장의 문서에서 찾으면 됩니다. **이 문서와 코드가 다르면 코드가 정본입니다.** 다른 점을 발견하면 알려 주세요.

---

0. 넘기기 전에 의뢰인이 할 일 (받는 분은 끝났는지만 확인)

> **알아 둘 것:** 이 Vercel 프로젝트는 GitHub와 연동되어 있어 `master`**에 들어간 커밋은 곧바로 운영 사이트에 배포됩니다.** 브랜치를 push하면 Preview 배포가 만들어질 수 있습니다. 0-2·0-3이 중요한 이유입니다.

#	조치	이유
0-1	✅ **완료(10/24)** `next` · `eslint-config-next` 16.4.0 → 16.4.2, 시험 382/382·typecheck·lint·build 통과, 운영 배포	16.4.0에는 `npm audit` critical 권고 2건(GHSA-2464-fhr4-w9xw, GHSA-673q-mjm6-r5xr)이 있었습니다. 남은 audit 경고는 high 4건이고 모두 같은 원인입니다: `@vercel/functions`가 간접으로 쓰는 옛 zod 4.1.11(메모리 고갈 권고). 이 앱은 그 경로로 사용자 입력을 검증하지 않습니다. 받는 분은 의존성을 바꾸지 않습니다
0-2	저장소 쓰기 보호. **권장안:** 7장 점검 뒤 저장소를 먼저 공개로 바꾸고 → `master` 브랜치 보호(PR 필수, 강제 push 금지)와 태그 `S*` 보호를 건 다음 → 받는 분은 collaborator가 아니라 **fork + PR**로만 작업	비공개 저장소는 GitHub 무료 요금제에서 보호 규칙을 걸 수 없습니다(API 403 확인). 봉인 태그 S3~S9가 지워지거나 `master`가 덮어써지면 봉인 검증이 깨집니다. **대가:** 일찍 공개하면 보고서 원고·콘티·KEIT 문의 초안·이 문서가 마감 1주 전부터 공개됩니다(공모전 규정상 문제 여부 미확인). 받아들일 수 없으면 GitHub Pro로 비공개 상태에서 보호를 겁니다
0-3	Vercel **Preview** 환경에서 `JUDGE_ACCESS_CODE` · `ANTHROPIC_API_KEY` · `KV_REST_API_URL` · `KV_REST_API_TOKEN`을 모두 지움	지금은 네 값 모두 Preview에도 있습니다. 브랜치 배포의 서버 코드를 바꾸면 값을 읽거나 운영 예산 카운터를 건드릴 수 있습니다. 지우면 Preview는 재생 모드로만 돕니다
0-4	「끊겼」 예외(4.2절 24번) 확정	알림 사실 문장이 이미 이 구문을 씁니다. 받는 분은 기본안대로 진행합니다
0-5	✅ **완료(10/24)** `.gitignore`에 `app/AGENTS.md` 추가	`npm run dev`를 실행하면 자동으로 생기는 파일입니다. 파일 본문은 커밋을 권하지만 이 저장소에서는 커밋하지 않습니다
0-6	✅ **완료(10/24)** `.gitattributes`에 `spec/** -text` 추가	Windows 기본 설정(`core.autocrlf=true`)으로 복제하면 `spec/*.json`이 CRLF로 바뀌어, 아무것도 고치지 않아도 바이트 비교 시험 2건

#	조치	이유
		(`materialize.test.ts`) 이 실패합니다
0-7	✅ **완료(10/24)** 자동 검사 체계와 GitHub Actions CI(6.1절)	받는 분의 PR마다 자동으로 돌아, 불변 조건(4.2절)을 깨뜨리면 병합 전에 빨간 불이 뜹니다
0-8	**PR #1(`fix/screen-defects`) 확인 후 병합** → 운영 배포 → `npm run lhci:prod`	검사를 만들며 찾은 기존 화면 결함 6건 수정(6.4절). 병합해야 받는 분이 결함 없는 상태에서 시작합니다

---

1. 3분 요약

무엇을 만들었나. 정전이 나면 인공호흡기·산소발생기를 쓰는 집마다 *배터리가 먼저 끝날지, 전기(또는 비상전원)가 먼저 올지*를 계산합니다. 그 결과로 담당자에게 처리 순서와 맞는 비상전원을 제안하는 운영자 도구입니다. 배정은 담당자가 승인해야만 이뤄지고, 119에는 위치·기기·예상 안전시간 세 가지만 넘어갑니다.

왜 필요한가. 공개된 정전 복구·순환단전 절차에는 "어느 집이 먼저 꺼지는지" 계산하는 단계가 없습니다. 인공호흡기 내장 배터리는 짧으면 1시간인데, 정부 집계로 최근 3년 아파트 정전 382건 중 55건이 12시간을 넘겼습니다.

AI(Claude)가 하는 일

1. **라벨 판독(Sonnet):** 기기 라벨 사진의 글자를 옮겨 적습니다. 코드가 사양 DB와 정확히 맞을 때만 기종 후보로 삼습니다. 그때도 담당자가 「기종 확인」을 하기 전까지는 이웃 기종 중 가장 짧은 값으로 보수적으로 계산합니다.
2. **보호자 알림 글(Sonnet):** 담당자가 결정한 내용을 보호자용 쉬운 글로 씁니다. 두 검사를 모두 통과해야 후보가 되고, 아니면 코드가 만든 고정 문안을 씁니다.
   - 1차: 규칙 검사
   - 2차: 다른 AI(Claude Opus)가 글만 읽고 뽑은 주장을 코드가 결정 기록과 대조

계산·순서·배정은 AI가 아닌 규칙 코드가 하고, 결정은 사람이 합니다.

검증은 이렇게 했습니다.

- **사전등록과 봉인.** 예측·판정 기준·코드 해시를 봉인 실행 전에 커밋했습니다. S3는 sha256 매니페스트, S4~S9는 OpenTimestamps 비트코인 증명입니다.
  - 판정한 사전등록 59건(미검정·판정 불가 2건, 구조 확인 1건 제외)은 두 갈래로 나뉩니다.
    - 실행 전 예측 22건: 18건 맞힘
    - 개발 시드 예비 실행·사전 분석 뒤 정한 '확인' 37건: 35건 맞힘
  - S8(레드팀이 같은 효과를 먼저 본 뒤 씀)과 S9(개발셋을 보고 정한 문턱)는 전부 '확인' 쪽입니다.
  - **한계:** 모든 봉인에서 비트코인 블록 시각이 봉인 실행 시작보다 늦습니다. 순서의 근거는 git 커밋 시각과 OTS 캘린더 접수뿐입니다(README, `seal/CHANGELOG.md` '봉인 시각의 한계').
- **틀린 예측과 약점도 숫자로 공개합니다.**
  - 배터리 하한이 모의 실제보다 길게 나온 경우 14.9%(모의, 사전등록 밖 사후 집계)
  - 봉인 때 나간 알림 글 60건 중 21건은 봉인 뒤 결함 판정('적어도' 표현만 있는 8건은 기준 변경이라 제외, 합치면 29건)
- **적대적 검수(레드팀) 18라운드.** 마지막 라운드에서 치명·중대 0건으로 끝났습니다. 그 뒤에 반영한 경미 수정은 별도로 검수하지 않았습니다.

> **이 작업의 가장 중요한 원칙:** 화면은 얼마든지 좋아져도 됩니다. 다만 **숫자·공개된 한계·AI 표시·사람 승인 구조는 약해지면 안 됩니다.** 심사에서 "과장하지 않고 한계까지 공개한다"가 이 작품의 신뢰 근거입니다. 이 요약을 영상·홍보 문구로 옮길 때도 단서·한계 문장을 함께 옮겨 주세요.

화면 지도

경로	누가 보나	내용
`/`	투표자(모바일)·심사위원	두 시계(배터리 대 복구), 30초 비교 위젯, 끝 카드
`/household`	데모	라벨 사진 → AI 글자 판독 → DB 대조 → 사람 확인 → 배터리 하한 분해식 → 보호자 대비 안내
`/ops`	심사위원(데스크톱)	운영자 화면: 모의 정전, 처리 순서, 승인·수정·확인 조작, 감사 로그, 승인 뒤 보호자 알림
`/ops/119`	데모	119에 넘어가는 3필드 화면
`/lab`	심사위원	리허설 랩: 같은 정전을 정책만 바꿔 반복한 결과. 절 번호 ①②③④⑥⑦⑧⑨(⑤ 없음), 화면 순서 ②③①④⑦⑧⑨⑥
`/ledger`	심사위원	AI 사용 장부(실측·비용), `#judge` 심사위원 3분 경로
`/about`	공통	역할·법적 고지·국외 전송·착안 공개(`#inspiration`)

2. 일정

오늘은 10/23(금)입니다. 영상은 **바뀐 디자인이 배포된 사이트**에서 녹화하므로 순서는 디자인 → 검수 → 영상입니다. 날짜는 의뢰인 제안이니, 어렵다면 **착수일에 바로** 알려 주세요.

날짜	받는 분	의뢰인	확인 지점
10/23(금)~24(토)	환경 준비, 현재 화면 진단, **디자인 방향안**(화면별 before/after 스케치 또는 시안)	0장 조치	**10/24 밤: 방향안 승인.** 구조 개편은 이 승인 없이 착수하지 않음
10/25(일)~27(화)	디자인 구현, PR	KEIT 답 반영 문구 전달(오면)	**10/27 밤: 디자인 동결.** 4.4절 통과 PR
10/28(수)	디자인 검수 1라운드, 지적 수정	**병합 전 AI 상태 화면 확인**(6.1절) → 병합·운영 배포 → 녹화 전용 코드 발급(5.3절)	배포본 검수 치명·중대 0
10/28(수)~29(목)	영상 녹화·편집 초안	초안 확인	**10/29 밤: 영상 초안 + 영상 검수 1라운드**
10/30(금) 오전	지적 수정, **영상 최종본 + 검수 보고서 납품**	수정본 교차 확인, 녹화 코드 회수(5.3절), 제출(8장)	제출
10/31(토)	(대기)	예비일	

늦어질 때의 우선순위: 영상 > 검수 > 디자인. 디자인이 10/27까지 끝나지 않으면 끝난 화면까지만 반영하고, 나머지는 현재 디자인으로 둔 채 녹화합니다.

3. 작업 환경

3.1 준비

- Node.js 24(의뢰인 환경 v24.14.1), Git, Python 3(6.1절 숫자 대조용)이 필요합니다.
- 앱은 저장소의 `app/` 폴더에 있습니다. Next.js 16, React 19.3, Tailwind CSS 4를 씁니다.
- ⚠️ Next.js 16은 예전 버전과 API·규칙이 다릅니다. 코드를 쓰기 전에 `app/node_modules/next/dist/docs/`의 해당 안내를 확인하세요.
- **의존성(`package.json`·lock)은 바꾸지 마세요.** 필요하면 먼저 상의해 주세요.

```
# 저장소를 fork한 뒤. Windows는 줄바꿈 변환을 끄고 받으세요(0-6으로 spec/는 해결됐지만 권장)
git clone -c core.autocrlf=false https://github.com/<본인 계정>/power-safety-net-ai.git
cd power-safety-net-ai/app
npm ci
npm run dev          # http://localhost:3000
```

3.2 API 키 없이 작업합니다

API 키·저장소 토큰·심사용 접근 코드는 **드리지 않습니다.** 코드는 예산 저장소(Upstash) 설정이 없으면 AI 실시간 호출 대신 **재생 모드**로 동작하게 만들어져 있습니다(`app/src/lib/claude/budget.ts`, `app/src/app/api/vision/route.ts`, `app/src/app/api/notice/route.ts`).

기능	로컬(설정 없음)에서
`/`, `/lab`, `/ledger`, `/about`, `/ops`, `/ops/119`	그대로 동작(AI 호출 없음)
`/household` 샘플 6장	봉인 측정 때 기록한 AI 출력 재생(원래 호출 0)
`/household` 사진 업로드	재생 모드 안내
`/ops` 보호자 알림 "AI로 쉬운 글 쓰기"	늘 고정 문안. **AI 글 상태(「AI 생성」 표지, 「실제 발송 기능 없음」 문장 등)는 로컬에서 볼 수 없습니다** → 6.1절

3.3 브랜치와 배포

- fork에서 `design/…` 브랜치로 작업하고 PR로 올립니다.
- **금지:** `master`에 직접 push, 강제 push, 태그 삭제·이동(`git push --force`, `git push --delete`, `git tag -d`). 봉인 태그 S3~S9가 깨집니다(7장).
- 의뢰인이 PR을 `master`에 병합하면 운영 사이트에 자동 배포됩니다. 확인은 로컬 `npm run build && npm start`(프로덕션 빌드)로 하세요.
- 커밋 메시지와 PR 설명은 한국어로, 무엇을 왜 바꿨는지 알 수 있게 써 주세요. 커밋 작성자 이메일은 공개되므로 원하면 GitHub noreply 주소를 쓰세요.

---

4. 작업 ① 디자인 업그레이드 (구조 개편 허용)

4.1 목표와 범위

- **목표:** 심사위원·투표자가 첫 10초 안에 "배터리 하한과 복구 시간을 비교해, 먼저 꺼질 집을 담당자가 먼저 챙기게 하는 도구"라고 이해하고, 믿을 만한 공공 서비스로 느끼게 합니다.
- **허용:** 색·타이포·간격·레이아웃·반응형, 화면 안 정보 배치와 흐름, 컴포넌트 분리·재구성, 전역 내비게이션 추가, 표·차트의 시각 표현 개선, 문구의 표현 다듬기(4.2절 범위 안에서).

- **대상 파일:**
  - `app/src/app/**`의 페이지·레이아웃(`page.tsx`, `layout.tsx`)
  - `app/src/components/**`
  - `app/src/app/globals.css`(디자인 토큰은 여기의 `@theme`)
  - `app/public/`(단, `samples/` 제외)
- **범위 밖(수정 금지):**
  - `app/src/app/api/**`(서버: 동의 검사·접근 코드·예산)
  - `app/src/lib/**`(계산 엔진·배정 규칙·AI 검사·고정 문안)
  - `app/src/data/**`(실측 수치)
  - `app/public/samples/`(봉인 측정 AI 출력과 짝인 라벨 이미지. `samples.json`이 경로로 참조하므로 압축·교체·이름 변경 금지)
  - `app/next.config.ts`(보안 헤더), `app/package.json`·lock, `app/eslint.config.mjs`, `app/vitest.config.mts`
  - `app/scripts/`, `tools/`, `spec/`, `prereg/`, `seal/`, `out/`, `datasets/`
  - **검사 체계**: `app/e2e/**`(시험·`axe-baseline.json`), `app/scripts/lint-claims*`·`claims-rules.json`·`check-*`, `app/playwright.config.ts`, `app/lighthouserc*.json`, `app/src/lib/ops/approval-scan.test.ts`, `.github/**`. 이 파일을 바꾼 PR은 CI의 '검사 체계 변경 확인'이 막고, 의뢰인이 내용을 보고 `test-change-approved` 라벨을 붙여야 통과합니다
- 범위 밖 파일을 고쳐야 할 것 같으면 **멈추고 먼저 물어 주세요.**
- **`data-testid` 속성은 지우거나 이름을 바꾸지 마세요.** 시험이 화면 요소를 찾는 표지입니다. 구조를 바꿔 요소가 옮겨 가면 같은 `data-testid`를 새 요소에 붙여 주세요.

4.2 불변 조건

화면을 다시 짜도 아래는 그대로여야 합니다. 어기면 검수에서 치명·중대로 판정합니다(6.3절).

A. 숫자와 문장 선택

1. 측정 수치는 `app/src/data/evidence.json`·`lab.json`에서 읽습니다(`app/src/lib/evidence.ts`, `lab.ts` 경유). **숫자를 화면 코드에 새로 직접 쓰지 마세요.** 값이 없으면 「측정 전」으로 바뀌는 구조를 유지합니다.
2. 화면 코드에 이미 있는 **숫자 포맷·파생 계산과, 어떤 문장을 보일지 고르는 조건문은 바꾸지 마세요.** 옮길 수는 있지만 같은 입력에 같은 문자열이 나와야 합니다. 대표적인 곳:
   - `ClosingCard.tsx`: `pct()`, 관문 통과일 때만 비교 문장, 유의성에 따라 "더 줄고/늘고/차이 없고"
   - `app/src/app/page.tsx`: 14.9% 계산(`toFixed(1)`), "N건 중 1건" 계산
   - `app/src/app/lab/page.tsx`: `aiClear` 조건, `effLo > 0` 조건
3. 숫자 정본은 매체마다 다릅니다.
   - **사이트:** evidence.json 표기(예: 15.0%).
   - **영상·보고서:** 콘티·보고서 원고 표기(예: 콘티의 15%, 73%, 60%). 둘 사이의 소수 자릿수 차이는 허용합니다.
   - 그 밖의 반올림·바꿔 쓰기(예: 56/60 → "93%")는 하지 마세요.

B. 공개된 한계와 단서 — 지우거나, 사실상 안 보이는 곳(접힌 영역 깊숙이, 아주 작은 글씨)으로 옮기지 마세요. 위치는 옮겨도 되지만 **같은 화면에서 읽혀야** 합니다. 4. 랜딩: 하한 14.9%가 실제보다 길게 나왔다는 문장, 보호자 알림 소개의 「시연용·담당자 확인 뒤 발송, 실제 발송 기능 없음」(`page.tsx` 아래쪽). 5. 보호자 알림 패널(`NoticePanel.tsx`): 「AI 생성」 표지, 검사 통과·담당자 확인 필요 표지, 「실제 발송 기능 없음」 문장. **이 셋은 운영자 화면에 AI 글이 떴을 때만 나타납니다.** 고정 문안 상태에서는 없는 것이 정상입니다. 알림 문장 자체(고정 문안, 꼬리말 「계산값이라 더 일찍 떨어질 수 있습니다」·「시연용 안내입니다」)는 `lib/notice/`가 만들므로 손대지 않습니다. 6. 랩·장부: 「모의」 「가정」 「봉인 시험셋」 한정어, 단순 규칙과 차이 없음, 틀린 예측, 나간 글 결함 21/60(+8건 단서), 맞바꿈(3·8시간급은 늘어남). 7. 위젯 표지 「가정 시나리오 범위 · 한전 실시간 예측 아님」과 두 시계 막대 라벨 「목표 상한까지 남는 차이 · 내장배터리만, 외장·이송 없을 때의 단순 비교」. **막대에 시간 숫자를 쓰지 않습니다**(출처 없는 파생 수치라서).

C. AI 표시와 법적 고지 (AI기본법 제31조, 의료기기 오인 방지) 8. 법정 배너(`components/LegalBanner.tsx`)

- **모든 페이지 상단 고정**, 문구는 코드 그대로입니다. 명세 1.1절 문구는 옛 판이고 **코드가 정본**입니다.
- 색·높이는 바꿔도 됩니다. 배너는 약 640~690px 폭(글꼴에 따라 다름)에서 두 줄이 됩니다. 높이를 고정해 문구를 자르지 마세요(자르면 배너 누락으로 치명).
- 배너 높이에 묶인 머리줄이 두 군데 있습니다: `/ops` 상단 바(`OpsScreen.tsx`), `/lab` 탭 바(`lab/page.tsx`). PR #1 병합 뒤에는 둘 다 배너 실제 높이 변수 `--banner-h` (`components/BannerHeightSync.tsx`)를 씁니다. 새 고정 머리줄을 만들면 같은 변수를 쓰세요. 겹침은 E2E `layout.spec.ts` 가 8개 폭에서 확인합니다.

9. AI 출력에는 `<AiTag>` (「AI 추출」·「AI 생성」) 표지를 붙입니다. AI가 아닌 계산 결과에 AI처럼 보이는 꾸밈(반짝이 아이콘, “AI 분석 중…” 등)을 넣지 마세요.
10. `/household` 업로드: 체크박스 2개(합성 라벨 / 국외 전송 동의)를 모두 해야 업로드가 열리고, 국외 전송·보관 기간 안내를 유지합니다.

D. 사람 승인 구조와, 화면 컴포넌트 안에만 있는 로직 — 아래 로직은 lib가 아니라 **화면 파일 안에만** 있습니다. 재구성할 때 같이 옮기지 않으면 조용히 사라집니다. 11. 배정 승인은 사람이 버튼을 눌러야 합니다. 자동·일괄 승인(“추천대로 모두 승인”) UI를 추가하지 마세요. 12. `status: "APPROVED"` 대입은 **`components/ops/ApproveDrawer.tsx` 한 곳에만** 있어야 합니다. - 시험(`lib/ops/approval-scan.test.ts`)이 경로까지 검사하므로 **이 파일의 이름·위치를 바꾸지 마세요.** - 시험은 큰따옴표 정규식만 보므로, 다른 방식(상수·작은따옴표)으로 승인 상태를 만드는 코드도 쓰지 마세요. 13. `ApproveDrawer.tsx` 에만 있는 로직 - 승인할 배정이 없는 레인(수동검토·대기·미배정)의 **승인 버튼 비활성.** 세션 코드에는 막는 로직이 없습니다. - 수정 저장의 **사유 코드 + 자유문 5자 이상** 검사(`ModifyForm`) - 재검토 대안 승인 경로(수정 + 재검토 감사 기록) - 확인 조작 숫자 입력의 `parseOperatorNumber` 검사 - 재신고 **빈칸 거부.** 이 검사가 빠지면 빈 입력이 0%로 재신고됩니다. - **보호자 알림 차단(`DecidedNotice`).** 119 표시분을 파기한 가구는 「보호자 알림 없음」, 재검토 표시 가구는 「보호자 알림 보류」입니다. 14. `OpsScreen.tsx` 에만 있는 로직 - 수동검토 행을 처리 순서 표 맨 위에 고정 - 「한전 복구 예정 통보」 입력 검증(0보다 큰 값, 하한 0) 15. `NoticePanel.tsx` : 사실이 바뀌면 이전 AI 글을 버리고, 가구 화면에서는 검사에 탈락한 원문을 숨깁니다. 16. `HouseholdScreen.tsx` - **업로드 이미지 축소 `shrink()` .** 캔버스로 다시 그려 EXIF(촬영 위치 등)를 지우고, 1568px로 줄이며, 2MB 상한을 겁니다. 업로드를 다시 짜면서 파일을 바로 보내면 위치정보가 국외 API로 나갑니다. - 「이 기종이 맞습니다」 확인 뒤에만 하한을 계산합니다. 17. `OpsProvider` - `app/src/app/ops/layout.tsx` 에 있어야 `/ops` 와 `/ops/119` 가 같은 상태를 씁니다. - 5분 단위 재계산 주기(대기 시한 만료가 여기에 달려 있음)를 유지합니다. - 상태는 탭 메모리에만 있어서, 새로고침·주소창 입력· **`/ops` 밖으로 나가는 이동**(전역 내비게이션 포함)이면 초기화됩니다. 의도된 동작이지만, 전역 내비게이션을 넣는다면 `/ops` 화면에서 “나가면 모의 정전이 초기화됩니다” 안내를 고려해 주세요. 18. `/ops/119` 는 정확히 3필드(위치·기기·예상 안전시간)입니다. 승인 전·파기 뒤에는 빈 화면이고, 배정 제안 승인 행은 나오지 않습니다. 19. 「재검토 필요」 배지, 「AI 판독 기종 미확인」 표시와 「기종 확인」 뒤 해제, 감사 로그를 유지합니다(「AI 판독 기종 미확인」은 현재 시연 프리셋에 해당 가구가 없어 화면에 나오지 않습니다. 코드만 보존하면 됩니다). 20. 심사용 접근 코드 입력칸은 `type="password"` 를 유지합니다. 21. `ClosingCard.tsx` 의 `robustPhrase` 함수는 시험이 import하므로 이름·위치를 유지합니다. 22. 다른 화면·콘티가 참조하는 앵커와 절 번호를 유지합니다. - 앵커: `/ledger#judge` , `/about#inspiration` , `/lab` 의 `#notice` · `#prereg` · `#compare` - `/lab` 절 번호(①②③④⑥⑦⑧⑨). 다른 곳에 “리허설 랩 ⑨”, “랩 ⑥” 같은 참조가 있습니다. 23. 서버 컴포넌트인 `app/src/app/page.tsx` , `app/src/app/lab/page.tsx` 를 `"use client"` 로 바꾸지 마세요. 엔진·데이터가 브라우저 번들에 들어갑니다.

E. 문구 규칙 (명세 `docs/spec.md` 9절). **새로 쓰거나 옮기면서 고친 문장에 적용합니다.** 기존 문장이 아래 24~27번에 걸리면 고치지 말고 의뢰인에게 보고만 하세요(단서 문장을 “고치다가” 약하게 만드는 일을 막기 위해서입니다). 알려진 기존 사례(같은 문장에 ‘모의’ 없음, 대개 절 제목·표지에 ‘모의’가 있음):

- 「현행 절차보다 공백 가구·시간 15.0% 감소」 — `ClosingCard.tsx` , `lab/page.tsx`
- 「합산하면 가까운 순이 0.6% 낫습니다」 — `ClosingCard.tsx` , `lab/page.tsx`
- 「전체 공백 가구·시간은 무작위보다 0.8% 더 줄고」 — `ClosingCard.tsx`
- 「무작위보다 공백 가구·시간이 0.8%… 더 줄었습니다」 「실측 판독 오류를 넣어도 … 17.2%… 줄였습니다」 — `lab/page.tsx`
- 「도구는 공백 가구·시간을 16.5% 줄였지만」 「14.3% 추가 감소」 — `lab/page.tsx`

- 요약 상자 「5.3~24.8% 더 줄고」 — `lab/page.tsx`

24. 금지 구문: 「생명을 구」 「생명을 지킵」 「안전을 보장」 「안전 보장」 「사망을 막」 「사망 예방」 「인명 구조」 「구조합니다」 「끊겼」 「살렸」 「골든타임을 지키」 「골든타임을 확보」 「골든타임 확보」 「막았」 「저장되지 않습니다」 「예측합니다」 「정확히 예측」 「검증됨」 「검증된 안전」 「KPI 달성」 「환자 상태 모니터링」 「실시간 모니터링」 「세계 수」 「1만 개 세계」 「페르소나」 「검토 지수」 「취약점 카드」, 범위 없는 「정확도 n%」.
    - **예외(기본안, 0-4에서 확정):** 「끊겼」은 정전 사실을 알리는 문장 안에서 허용합니다. 해당 위치는 알림 고정 문안 「정전으로 {기기} 전원이 끊겼습니다」(`lib/notice/rules.ts`), `/lab` ⑨의 봉인 AI 글 예시, 랜딩 대체 문장(`page.tsx`)입니다. 성과·효과를 주장하는 문장에서는 금지이고, 새로 쓰는 문장에는 넣지 마세요.
25. %가 들어간 성과 문장에는 같은 문장 안에 「모의」나 「시뮬레이션」을 넣습니다(라벨 판독은 「봉인 시험셋」).
26. 분모가 20 미만이면 %를 쓰지 않고 「9칸 중 3칸」처럼 씁니다.
27. 모의 결과를 '실측'이라고 부르지 않습니다. '실측'은 봉인 시험셋 측정, 지연·비용, 그리고 입력 자료가 실측임을 밝힐 때(예: 「실측 정전 분포 모의」, 「실측 판독 오류를 넣은 모의」)에만 씁니다. 명세 9절은 영상 콘티 기준이고, 이 문서는 사이트에 이미 있는 입력 자료 표기를 허용하도록 넓혀 적었습니다.
28. 대기 표기는 「대기(재확인)」입니다.
29. 랜딩 첫 화면(배너 제외)에는 'AI', '시뮬레이션', '예측', '생명', '안전'을 쓰지 않습니다. 지금은 390×844에서 위젯 윗부분만 첫 화면 하단에 걸치고, 위젯 표지(「…예측 아님」)는 첫 화면 밖입니다. 위젯을 올려 표지가 첫 화면에 들어오면 이 규칙과 충돌합니다.

F. 차별화(착안 출처와 겹치지 않기) 30. 3D, '세계 수' 선택기·카운터, 페르소나 군중, 취약점 카드, 검토 지수 가중치, LED·LCD 표현은 쓰지 않습니다. 결과는 표와 2D 차트로만 냅니다. 이 작품은 다른 해커톤 출품작의 실험 틀에서 착안했고 그 사실을 공개하고 있어, 화면 문법이 겹치면 안 됩니다.

G. 품질 기준 31. 랜딩은 390×844에서 **스크롤 없이** 배너·H1·두 시계·CTA가 보입니다(현재 충족). 두 시계는 CSS 애니메이션이고, `prefers-reduced-motion`이면 정지 그림입니다. 32. 명도 대비 4.5:1 이상, 키보드로 모든 조작 가능. 랜딩 본문은 16px 이상입니다. 다른 화면의 기존 크기(표 12~14px 등)보다 작아지지 않게 하고, 키울수록 좋습니다. 33. 무거운 애니메이션·차트·UI 라이브러리를 새로 넣지 마세요(의존성 금지). 랜딩 초기 JS는 **예산 안**이어야 합니다. 명세의 120KB 목표는 이미 대체되었습니다. 기준선은 gzip **179,753B**(10/24, Next 16.4.2, JS 8개, 대부분 Next 기본 런타임, PR #1 뒤 179,917B)이고, 예산(`check:budget`)은 약 3% 여유를 둔 **185,000B**입니다. - 측정: `npm run build && npm run check:budget`. `/` 빌드 HTML에 실린 `<script src>`만 더하고, 링크 미리 받기로 따라오는 다른 페이지 청크는 제외합니다(전역 내비게이션을 넣어 그 청크가 늘어나는 것은 정상). 34. 외부 글꼴·CDN·이미지 호스트를 새로 쓰면 국외 전송 고지 범위가 달라질 수 있으니 먼저 상의하세요. 지금 글꼴 체인은 `"Pretendard"` → 시스템 한글 글꼴입니다(`globals.css`의 `--font-sans`, 웹폰트 파일은 싣지 않음). Pretendard가 없는 PC에서는 맑은 고딕으로 보입니다. **녹화 PC에서 보이는 글꼴이 의도한 글꼴인지** 확인해 주세요.

4.3 개선 기대 지점 (의뢰인 의견, 강제 아님)

- 지금 디자인은 기능 위주의 단순한 스타일입니다. 공공 서비스다운 신뢰감과 정보 위계를 원합니다.
- `/lab`이 길고 빽빽합니다. 목차·요약 상자·절 사이 이동을 정리해 주세요. 절 번호는 유지합니다(22번).
- `/ops`는 표·서랍·감사 로그가 한 화면에 몰려 있습니다. 처리 순서 → 승인 → 119 흐름이 눈에 따라오게 해 주세요.
- 페이지 사이 이동(전역 내비게이션)이 약합니다(17번 주의).
- 공유 미리보기(OG 이미지 1200×630)가 없습니다. 선택 과제이고, 정적 이미지 파일로 넣으면 됩니다.

4.4 디자인 PR을 올리기 전 통과 조건

```
cd app
npx playwright install chromium   # 처음 한 번
npm run check:all                 # 타입·린트·단위(커버리지)·빌드·문구·번들·JS 예산·숫자 대조·E2E 전부
```

- `check:all`이 통과하고, PR의 GitHub Actions(ci)가 녹색이어야 합니다(6.1절).

- **시험을 고쳐서 통과시키지 마세요.** 디자인이 바뀌어 시험의 선택자(`data-testid` 등)를 옮겨야 하면 그렇게 하되, 시험이 확인하는 내용(무엇이 보여야 하고 무엇이 막혀야 하는지)은 그대로 두세요. 시험·`e2e/axe-baseline.json`·`scripts/lint-claims.allow.json`을 바꿨다면 PR 설명에 이유를 적어 주세요.
- 6.2절 수동 체크리스트(자동화하지 못한 항목)를 직접 확인하고 결과를 PR 설명에 붙여 주세요.
- 화면별 before/after 스크린숏을 붙여 주세요(390px, 768px, 1440px).
- 4.2절 D의 로직을 옮겼다면, 어느 파일로 옮겼는지 PR 설명에 표로 적어 주세요.
- 4.2절 E에 걸리는 기존 문장을 발견했다면 목록으로 적어 주세요(고치지 않음).

---

5. 작업 ② 3분 시연영상

5.1 기준 콘티

`submission/video_storyboard.md`(v4)가 기준입니다. 장면 순서·시간 배분·내레이션 문안이 들어 있습니다.

- **방식은 재량입니다.** 녹화 도구, 내레이션(본인 목소리·성우·TTS), 자막 스타일, 편집 툴을 자유롭게 고르세요. TTS나 생성형 음성을 쓰면 영상 끝 자막에 그 사실을 밝혀 주세요. 이 작품은 AI 역할 표기를 지키는 것이 원칙입니다.
- 장면 구성은 바꿔도 됩니다. 단, 내레이션·자막의 **숫자와 단서 문장은 콘티 그대로** 씁니다. 바꾸려면 의뢰인 확인을 받으세요(정본 규칙은 4.2절 3번).
- 콘티의 '심사용 코드로 녹화' 메모는 이 문서 5.3절로 대체합니다.

5.2 필수 조건

1. **길이 3:00 이하**, 1920×1080. 첫 10초와 마지막 10초가 훅입니다.
2. **운영 사이트(배포본)**에서 녹화합니다. 로컬 화면이나 합성 목업은 쓰지 마세요.
3. 법정 배너가 **모든 장면에** 보여야 합니다.
4. `/ops` 승인 장면은 **사람이 승인 버튼을 누르는 모습**으로 찍습니다.
5. 콘티의 공개 단서(14.9%, 21/60, 보완 위반 7건)를 빼지 마세요.
6. 4.2절 E가 내레이션·자막에도 적용됩니다(「끊겼」 예외 포함). %문장에는 「모의」를 넣고, 모의 결과에 '실측'을 쓰지 마세요.
7. 실제 환자·실제 기기 사진·실제 인물·실제 기관 로고를 넣지 마세요. 화면의 가구·자원은 모두 합성 데이터입니다.
8. 음악·효과음·스톡 소스는 상업 이용이 가능한 라이선스만 쓰고, 출처 목록을 납품물에 포함합니다.

5.3 녹화 흐름 주의

`/ops` 상태 유지

- 상태는 탭 메모리에만 있습니다. `/ops` ↔ `/ops/119`는 화면 안 링크로만 이동하세요.
- 새로고침, 주소창 입력, `/ops` 밖으로 나가는 이동을 하면 처음부터 다시 해야 합니다.

기본 시나리오(수원형)에서 보이는 것

- 정전 직후 레인 분포는 대략 다음과 같습니다: 수동검토 6, 공백 단축 배정 5, 쉼터 이동 3, 미배정 5, 119 11. **「배정 제안」과 「대기(재확인)」는 0건입니다.**
- 이 둘을 보이려면 상단 **「한전 복구 예정 통보」에 1(시간)을 입력**합니다. 수원형 기준으로 배정 제안 1건, 대기(재확인) 6건이 생깁니다.

알림 장면(1:15–1:35)

- 파기하지 않은 「공백 단축 배정」이나 「쉼터 이동」 가구를 승인한 뒤 그 가구의 알림을 엽니다.
- 콘티 0:35–1:15에서 119 표시분을 파기한 가구는 「보호자 알림 없음」으로 막힙니다(4.2절 13번).

AI 실시간 장면(녹화 전용 접근 코드, 의뢰인이 진행)

1. 의뢰인이 22자 이상 무작위 값을 만들어, Vercel production의 `JUDGE_ACCESS_CODE`에 **쉼표로 덧붙이고 운영 재배포**합니다. 환경변수는 재배포해야 반영되고, 22자 미만 값은 조용히 무시됩니다.
2. 값을 받는 분에게 따로 전달합니다. **심사용 코드 자체는 어떤 경우에도 전달하지 않습니다.**
3. 받은 값은 문서·저장소·채팅 기록·영상 어디에도 남기지 마세요. 입력칸은 가려지지만, 붙여넣기 미리보기·클립보드 도구 창이 화면에 비치지 않게 주의하세요.
4. **한 번에 정확히 붙여 넣으세요.** 같은 IP에서 시간당 5회 넘게 틀리면 그 IP는 1시간 막힙니다.
5. 녹화 코드도 심사용 예산(하루 상한 포함)을 씁니다. 다시 찍을 때마다 심사위원 몫이 줄어드니, 리허설은 고정 문안 상태로 하고 실시간 생성은 실제 녹화에만 쓰세요.
6. 녹화가 끝나면 의뢰인이 그 값을 지우고 **다시 재배포**한 뒤, 옛 값이 거부되는지 확인합니다(늦어도 10/30).
7. 한도에 걸리면 콘티 메모대로 `/lab` ⑨의 「봉인 측정 때 AI가 쓴 글」을 대신 보여 줍니다.

`/household`

- 콘티 0:10–0:35는 **샘플** 재생 장면이라 코드가 필요 없습니다.
- 업로드 장면을 추가하려면 합성 라벨 이미지(`datasets/labels/dev/*.jpg`)만 쓰세요.

5.4 납품물

파일	형식
최종 영상	MP4(H.264, 1920×1080, 3:00 이하)
자막 원문	SRT 또는 TXT(숫자 대조용)
내레이션 원고(최종)	TXT 또는 MD
사용 소스 목록	음악·효과음·글꼴·음성 도구의 출처와 라이선스
편집 프로젝트 파일	재편집 대비(가능하면)

---

6. 작업 ③ 검수

검수는 **디자인 PR(10/27~28)**과 **영상 초안(10/29)**에 각각 한 라운드씩 하고, 지적을 고친 뒤 재검수합니다. 만든 사람이 스스로 검수하는 구조라서, 최종본은 의뢰인이 6.2절 표본으로 한 번 더 교차 확인합니다.

6.1 자동 검사 (10/24 구축, PR마다 GITHUB ACTIONS에서 자동 실행)

한 번에 돌리기: `cd app && npm run check:all`(약 5분). PR을 올리면 `.github/workflows/ci.yml`이 같은 검사를 Linux에서 돌립니다(비밀값 없이 동작).

명령(`app/` 에서)	무엇을 지키나(인수인계서 4.2절)	10/24 기준값
`npm run typecheck`	타입 오류	0
`npm run lint`	ESLint(경고도 실패) + **보호 규칙**: 승인 상태·승인 감사 값("APPROVED"·"APPROVE")은 `ApproveDrawer.tsx` 밖에서 만들 수 없음(12번, 상수·함수 반환·템플릿 포함, 타입 선언과 비교만 허용), 결정 경로(alloc·engine) 초월 함수·거듭제곱·`Math` 우회 금지, 화면·배정 규칙의 `WorldTruth` import 금지	0
`npm run coverage`	단위 시험 382건 + 커버리지 하한(전체 줄 79%, engine·alloc 줄 90%)	382/382, 전체 줄 79.2%, engine 98.5%, alloc 99.6%
`npm run build`	빌드	성공
`npm run lint:claims -- --require-build`	**문구**: 금지 구문(24번, 목록은 `scripts/claims-rules.json`), 범위 없는 「정확도 n%」, 한정어(25번: 랜딩·랩·장부의 % 성과 문장, 끝 카드, 콘티, 보고서 기대효과), 콘티의 '실측' 규칙(27번). 화면 코드 문자열·JSX 글자(인라인 요소로 쪼갠 문구와 `+`로 이은 문자열 포함, 주석 제외)·빌드 HTML·보고서 1부·콘티를 본다. 기존 문장 25개는 `scripts/lint-claims.allow.json`에 **문장 전체 일치**로만 예외	새 위반 0
`npm run check:bundle`	브라우저 번들에 API 키 형태·비밀 환경변수 이름 없음, 소스에 비밀처럼 보이는 `NEXT_PUBLIC_` 변수 없음	0
`npm run check:budget`	랜딩 초기 JS ≤ 180,500B(33번, 기준선 위 약 0.3%)	179,753B
`npm run check:report`	보고서 1부 숫자 전체 + 콘티 단서 4문구 = evidence.json	통과
`npm run test:e2e`	E2E 84건(데스크톱 1440 + Pixel 7) + 접근성(axe). 아래 목록	84/84(두 번 연속)
`npm run lhci`	Lighthouse(로컬): 접근성 점수 ≥ 0.9는 실패 조건, LCP는 경고만(로컬은 압축·CDN이 없어 느리게 나옴)	접근성 1.0, LCP 2.18초(경고)
`npm run lhci:prod`	**명세 예산 LCP ≤ 2.0초**를 운영 URL에서 판정(병합·배포 뒤 의뢰인이 실행)	1.86초 통과

E2E가 확인하는 것(`app/e2e/`):

- **모든 시험 공통**(고정장치): 가로채지 않은 `/api` 호출 0, 외부 호스트 요청 0(34번), 위치 권한 호출 0(조작 뒤 포함), 시험이 끝난 화면(서랍·알림·업로드를 연 상태)의 보이는 글자에 금지 구문 0
- **'보이는지' 검사**(`expectReadable`): DOM에 있는지만이 아니라 숨김·1px·덮임·12px 미만·대비 4.5:1 미만이 아닌지. 배너, AI 표지(추출·생성), 단서 문장(14.9%, 실제 발송 기능 없음, 막대 라벨)에 적용
- 전 페이지(7개): 배너 고정·문구·스크롤 뒤 덮이지 않음(8번), 6개 폭 가로 스크롤 없음, 앵커(22번), axe serious·critical 새 위반 0(기존 위반은 `e2e/axe-baseline.json`, PR #1 뒤 0건)
- 랜딩: 390×844 첫 화면 구성(31번), 첫 화면 금지 단어(29번), 두 시계 영역에 출처 없는 'n시간' 숫자 없음·라벨(7번), 동작 줄이기, 위젯 데이터 요청 0, 14.9% 문장의 **뜻**(모의·실제보다 긴 경우·사후 집계)과 「실제 발송 기능 없음」, 숫자 = evidence.json
- `/lab`·`/ledger`(`clues.spec.ts`): 절 번호↔앵커 짝(22번), 감소율·'단순 규칙과 차이 없음'·'가까운 순이 더 나음'·하한 과대예측 사후 집계·나간 글 결함 판정·맞바꿈·틀린 예측(×)·라벨 판독 정확·틀린 글 통과·'적어도' 제외분 합계를 evidence.json에서 다시 만들어 대조

- `/ops`: 수동검토 맨 위, 승인 전 119 빈 화면, 승인 막힌 3레인은 `disabled` 속성이고 **강제로 눌러도 승인 기록 0**(13번), 일괄·자동 승인 버튼 0개와 승인 한 번 = 승인 행 1·APPROVE 1건(11번), 119 3필드만·절대 시각·파기(18번), 배정 제안은 119에 없음, 수정 사유 경계(코드 없음·4자 거부, 5자 저장, 14번), 재신고 빈칸·공백·글자 거부, 재검토 배지·감사 행·알림 보류, 파기·재검토 가구는 알림 판 0개(15번), 한전 통보 0·음수·빈칸·글자 거부(14번), 고정 문안·「소리로 듣기」·코드 칸 가림(20번), **AI 알림 상태는 서버 응답을 흉내 내** 표지 3개와 다른 가구로 AI 글이 넘어가지 않음(5·16번), 심사위원 경로에서 클릭 없이 승인 0, ×600 대기 만료, 서랍 입력이 다른 가구로 이어지지 않음
- `/household`: 샘플 6장 호출 0·AI 추출 표지(보이는지 포함)·일치/수동검토·확인 전 분해식 없음(16번), 샘플을 바꾸면 확인을 다시 받음, 체크 2개(10번), **EXIF가 든 2400×1600 사진을 올려 요청을 가로채** EXIF 제거·긴 변 1568px 이하(16번), 2MB 상한, 검사 탈락 AI 원문 숨김(15번), 브라우저 저장소에 이미지 없음
- 배너 겹침: 8개 폭에서 `/ops` 상단 바·`/lab` 탭 바(8번)

실제 AI·예산을 쓰지 않습니다. E2E 고정장치(`e2e/fixtures.ts`)가 모든 `/api/**` 요청을 가로챕니다. 시험이 직접 응답을 지정하지 않은 호출이 하나라도 있으면 그 시험은 실패합니다. 서버도 예산 저장소 변수를 비운 채 띄웁니다.

검사가 막지 못하는 것(남은 한계): 시험은 정해진 시나리오와 문구를 확인합니다. 시험이 모르는 새 화면·새 문장의 뜻이 바뀌는 경우, 정적 분석을 일부러 피하는 코드(예: 문자열을 여러 조각으로 나눠 승인 상태를 만듦)는 사람이 봐야 합니다. 그래서 6.2절 수동 확인과 의뢰인 교차 확인을 남겨 둡니다.

시험 결과가 다르게 나올 때:

- `materialize.test.ts` 2건 실패: 줄바꿈 문제입니다(3.1절 복제 명령).
- 새 복제본에서는 단위 시험이 381 통과 + 1 skip으로 나옵니다. git 밖 연구 노트가 필요한 시험이라 정상입니다.
- 그 밖의 실패는 내 변경 탓으로 보고 원인을 확인하세요(간헐 실패 원인이던 시간 제한은 10/24 고침).

6.2 수동 체크리스트 (자동화하지 못한 것만)

화면

- ☐ Tab 키만으로 주요 버튼에 닿고 **포커스 표시가 보임**(axe는 포커스 표시를 판정하지 않음)
- ☐ 4.2절 E 소분모 규칙(26번): 분모 20 미만 비율을 %로 쓰지 않음. 어떤 %의 분모가 몇인지 글자에 없어 자동 판정이 불가합니다
- ☐ `/lab`·`/ledger`의 숫자 = `app/src/data/evidence.json`(무작위 10개 표본). E2E는 핵심 단서 숫자만 대조합니다(표 안의 숫자 전부는 아님)
- ☐ 단서·한계 문장이 '사실상 안 보이는 위치'(접힌 영역 깊숙이, 맨 아래 각주 등)로 밀리지 않았는지(B절). 숨김·덮임·작은 글씨·저대비는 검사가 보지만, 배치의 눈에 띄는 정도는 사람이 봅니다
- ☐ 외장배터리 확인 전 하한에 외장 0: 단위 시험(I3)이 계산을 지키고, 화면에서는 서랍 분해식으로 확인
- ☐ (코드 확인만) 「AI 판독 기종 미확인」 표시·해제 코드가 남아 있음(19번, 시연 프리셋에 해당 가구가 없어 화면에 안 나옴)
- ☐ 실제 AI 알림 한 번(의뢰인): E2E는 응답 모양만 흉내 냅니다. 병합 뒤 운영에서 심사용 코드로 1회 생성해 표지 확인

영상 (자막 자동 대조 없음)

- ☐ 3:00 이하, 1920×1080, 배너 상시 노출
- ☐ 자막·내레이션 숫자를 콘티 v4·evidence.json과 **전부 수동 대조**(4.2절 3번 정본 규칙). 자막 원문을 `submission/`에 넣으면 `lint:claims`로 문구 규칙도 볼 수 있습니다(파일명을 알려 주면 대상에 추가)
- ☐ 금지 구문 0(예외 제외), %문장 한정어, 모의 결과에 '실측' 없음
- ☐ 접근 코드·개인정보·실제 인물·로고 노출 없음
- ☐ 생성형 음성 사용 시 표기, 소스 라이선스 목록 있음

6.3 판정 기준과 보고 양식

- **치명:** 숫자 오류, 공개 단서 삭제, 배너·AI 표지 누락, 승인 우회, 키·코드 노출, EXIF 제거 유실, 새 금지 구문
- **중대:** 4.2절 D 로직 유실, 첫 화면 기준 미달, 화면 깨짐·겹침, 접근성 위반(대비·키보드), 앵커·링크 깨짐, 단서가 사실상 안 보이는 위치로 밀려남
- **경미:** 정렬·여백·오탈자 등
- **기존 결함**(작업 전부터 있던 것, 예: 4.2절 E의 기존 문장)은 등급을 매기되 '기존'으로 표시하고, 완료 판정에서 따로 셉니다.
- **완료 기준:** 마지막 검수 라운드에서 **새로 생긴 치명·중대 0건.** 남은 경미·기존 항목은 목록으로 넘깁니다.

`docs/REVIEW_<날짜>.md`에 다음 형식으로 남겨 주세요.

```
## 검수 라운드 N (YYYY-MM-DD, 대상: 커밋 해시 또는 배포 URL)
| # | 등급 | 새로/기존 | 화면·시각 | 무엇이 | 기대 | 근거(스크린숏 경로) | 상태 |
```

6.4 검사를 만들며 찾은 기존 결함 (10/24)

#	결함	처리
1	`/household` : 샘플을 바꿔도 앞 샘플의 「이 기종이 맞습니다」가 이어져, 확인하지 않은 기종의 하한이 보임(16번 위반)	PR #1에서 수정
2	`/ops` : 다른 가구를 열어도 서랍의 사유·수정 사유 입력이 남음	PR #1에서 수정
3	두 줄 배너(약 640~690px)가 `/ops` 상단 바·`/lab` 탭 바를 가림	PR #1에서 수정(`--banner-h`)
4	axe serious 31건: '사양 확인' 배지 대비 4.50:1, `/lab` 빈틈 지도 중간 진하기 칸 대비 최저 2.6:1, 수동검토 'DB 기종 선택' 상자 이름표 없음	PR #1에서 수정(0건)
5	단위 시험 간헐 실패: 전수 조합 시험(약 4.7초)이 기본 시간 제한 5초를 넘김	master에서 수정(시간 제한 30초)
8	`/ops` 재신고 칸에 공백만 넣으면 `Number(" ")`=0이라 0% 재신고로 저장됨(재검토·알림 보류까지 생김)	PR #1에서 수정
6	끝 카드·콘티의 % 문장 4개가 같은 문장 안에 '모의' 없음, /lab의 「막았습니다」 2곳(검사기 성능 설명)	**고치지 않음**(문구는 18라운드 결과라 의뢰인 결정으로 유지). 문구 검사 예외에 등록
7	로컬 빌드의 랜딩 LCP 2.18초(예산 2.0초 초과)	운영 측정 1.86초로 예산 안. 로컬은 압축·CDN 차이라 경고로만 감시

PR #1을 병합하기 전에는 1~3·8번 시험이 '실패가 정상'(`test.fail`)으로 표시돼 있고, 병합하면 일반 시험으로 바뀝니다. 병합 전 master에서 이 시험이 "예상과 달리 통과"로 실패하면, 그 결함이 다른 이유로 사라졌다는 뜻이니 알려 주세요.

---

7. 저장소 공개 전환 (의뢰인 결정: 공개)

공개 전환은 의뢰인이 합니다(권장 시점은 0-2). 받는 분도 검수 보고서에서 한 번 더 확인해 주세요.

점검	10/23 결과
git 전체 이력에 API 키 형태 문자열	0건
심사용 접근 코드 값이 이력·추적 파일에 있음	0건(코드는 git 제외 폴더 `submission/private/` 에만 있음)
`.env*`, `.vercel`, `submission/private/` 가 git에 올라감	아니오(`.gitignore` 로 제외)
커밋 작성자 이메일	모든 커밋에 의뢰인 이메일이 있어 공개 시 보임 → 받아들이거나, 앞으로의 커밋만 noreply로
공개되는 내부 문서	`submission/keit_inquiry_draft.md` , `submission/report_draft.md` 3부 '작성 메모', 이 문서 → 비밀은 없음

> ⚠️ **이력 재작성(filter-repo, rebase, force-push)과 태그 변경 금지.** 봉인 파일이 커밋 해시를 직접 담고 있고(`seal/S4.txt` 등), 원격에 S3~S9 태그가 있습니다. 이력이나 태그를 바꾸면 봉인 검증이 모두 깨집니다.

---

8. 의뢰인 쪽 남은 일 (참고용, 받는 분 작업 아님)

일	기한	받는 분에게 영향
0장 조치	넘기기 전	착수 조건
PR #1 병합 → 운영 배포 확인 → `npm run lhci:prod` (0-8)	넘기기 전	결함 없는 상태에서 착수
KEIT 문의 메일 발송(`submission/keit_inquiry_draft.md`)	10/23	답에 따라 사이트·보고서 문구가 바뀔 수 있음 → 의뢰인이 전달
병합 전 AI 상태 화면 확인(6.1절)	10/28	병합 조건
녹화 전용 코드 발급 / 회수(5.3절)	10/28 / 10/30	녹화 가능 여부
처음 보는 사람 3명 10초 시험	10/29	디자인 결과 확인
보고서 기입과 숫자 대조 `--final` (아래 절차)	10/30	없음
신청서 서명, 이메일 제출, 수신 확인 보관	10/30	없음
Vercel 환경변수 `VOTE_END_KST` 를 공고문의 심사·투표 종료일로(없으면 코드 기본값 12/31)	아무 때나	없음

보고서 기입 절차. `--final` 검사는 추적 파일 `submission/report_draft.md` 에 성명·접근 코드가 채워져 있어야 통과합니다(경로 고정).

1. 먼저 `report_draft.md` 의 다른 수정(KEIT 반영 등)을 커밋합니다. 3단계 되돌리기가 그 수정까지 지우기 때문입니다.
2. 추적 파일에 성명·접근 코드를 **임시로** 채우고 `python -I tools/check_report_numbers.py --final` 을 실행합니다.
3. Word·PDF를 만들어 **`submission/private/` 에** 저장합니다. `submission/out/` 은 git이 추적하는 폴더라 쓰지 않습니다.
4. 곧바로 `git checkout -- submission/report_draft.md` 로 되돌립니다.
5. 커밋·공개 전에 `git status --porcelain --untracked-files=all submission/` 출력이 비어 있는지 확인합니다. Word·PDF 안의 코드는 `git grep` 으로 찾을 수 없으므로, 파일이 추적 폴더에 생기지 않았는지를 이 명령으로 봅니다.

---

9. 연락과 판단 원칙

- **애매하면 멈추고 묻습니다.** 특히 다음 경우입니다.
  1. 4.2절 불변 조건과 충돌할 때
  2. 범위 밖 파일·의존성을 고쳐야 할 것 같을 때
  3. 숫자·단서 문장의 표현을 바꾸고 싶을 때
- 정해진 범위 안의 세부 실행(색 값, 간격, 컴포넌트 분리 방식, 편집 리듬)은 묻지 말고 진행하세요.
- 문제가 생기면 숨기지 말고 그대로 알려 주세요. 시험 실패, 일정 지연, 녹화 실패 모두 해당합니다. 이 작품은 한계를 그대로 공개하는 것이 신뢰 근거라서, 포장된 보고보다 정확한 보고가 쓸모 있습니다.
- 매일 저녁 한 줄 진행 보고(한 것 / 막힌 것 / 내일 할 것)를 부탁드립니다.

---

10. 용어

용어	뜻
하한(L)	배터리가 이만큼은 간다고 *계산한* 시간. 계산값이라 실제보다 길게 나올 수 있음(모의 14.9%)
공백	배터리가 끝난 뒤 전기·비상전원이 오기 전까지 기기가 멈춘 시간
레인	가구별 처리 갈래(배정 제안, 공백 단축 배정, 119, 쉼터 이동, 수동검토, 대기, 미배정)
수동검토	AI 판독이 DB와 정확히 맞지 않아 사람이 기종을 확인해야 하는 가구
대기(재확인)	지금은 배정하지 않고 시한까지 다시 확인하는 상태
봉인(S3~S9)	실행 전에 예측·기준·코드 해시를 커밋하고 타임스탬프를 받은 것(한계는 1장)
모의	합성 데이터 위의 시뮬레이션 결과(실제 정전 실측 아님)
봉인 시험셋	봉인 뒤 한 번만 측정한 AI 시험 자료(라벨 판독·알림)
고정 문안	AI 글이 검사를 통과하지 못할 때 쓰는, 코드가 만든 알림 문장
레드팀	결함을 찾으려고 적대적으로 검토하는 절차

11. 참고 문서 지도

알고 싶은 것	문서
프로젝트 개요·주요 결과 숫자	`README.md`
화면별 상세 요구(랜딩 1.3, `/ops` 1.4, `/ops/119` 1.5, `/household` 1.6)	`docs/spec.md` 1장(배너 문구는 코드가 정본)
금지 표현·한정어 규칙·시험 기준	`docs/spec.md` 9절
개발·레드팀 경과	`docs/DEVLOG.md`
봉인·사전등록 경과, 봉인 시각의 한계	`seal/CHANGELOG.md`
영상 콘티	`submission/video_storyboard.md`
성과보고서 원고	`submission/report_draft.md` (Word·PDF는 `submission/out/성과보고서_1부_v7.*`)
개인정보·국외 전송 고지	`docs/compliance/README.md`
화면 숫자의 출처	`app/src/data/evidence.json` (직접 편집 금지, `app/scripts/sync-evidence.ts` 가 생성)